

GBGHMVNNEXSONJEN - My First DevOps Project

The Complete Story in Simple Words (Even a 6-Year-Old Can Understand!)

Created by: afzal-baba | Date: Sep 2, 2026 | GitHub: github.com/afzal-baba/project-GBGHMVNNEXSONJEN

1. What is this Project? Think of a Toy Factory!

Imagine you have a TOY FACTORY.

GBGHMVNNEXSONJEN is a fancy name, but it's just a factory name. Let's break it:

- **G = GitHub**: Your toy drawing book (where you keep designs)
- **B & M = Maven**: The machine that builds toys from drawings
- **N = Nexus**: The big shelf where you store finished toys
- **SON = SonarQube**: The toy checker - checks if toy is broken or safe
- **JEN = Jenkins**: The boss robot who tells everyone what to do in order

So full story: You draw a toy (code) -> Keep in book (GitHub) -> Boss (Jenkins) says BUILD! -> Machine (Maven) builds -> Shelf (Nexus) stores -> Checker (SonarQube) says OK!

2. Where are all Files? (Like Treasure Map)

File / Folder	Location on Your Laptop	What it does? Simple meaning
Project Folder	E:\project-GBGHMVNNEXSONJEN	Your main factory building
Source Code	E:\project-GBGHMVNNEXSONJEN\src\main\java\com\mydrawing\sonar\Applc\World	Toy drawing on paper - Application
pom.xml	E:\project-GBGHMVNNEXSONJEN\pom.xml	Recipe book - tells Maven how to build + Nexus address
Jenkinsfile	E:\project-GBGHMVNNEXSONJEN\Jenkinsfile	Boss robot's To-Do list - 3 steps
Maven	E:\maven\apache-maven-3.9.14	The building machine
Maven Settings	C:\Users\AFZALBABA\m2\settings.xml	Machine's secret password to put toys on shelf
Java	C:\devops\java\jdk-21	Electricity for machines
Git Repo	E:\project-GBGHMVNNEXSONJEN\.git	Invisible history book of all drawings
Jenkins Home	C:\ProgramData\Jenkins\jenkins	Boss robot's brain and memory
Jenkins Workspace	C:\ProgramData\Jenkins\jenkins\workspace\GBGHMVNNEXSONJEN	Where Boss SONJEN builds toys

3. All Problems We Faced & How We Fixed (The Fun Part!)

Problem 1: pom.xml not found

What Jenkins/Git said: Maven said 'Where is recipe?'

Why (6yr old): We were running maven from E:\ drive, not from inside project folder.

How we fixed: cd /e/project-GBGHMVNNEXSONJEN then mvn clean package - like going inside kitchen before cooking!

Problem 2: GitHub push failed - remote contains work

What Jenkins/Git said: GitHub said 'I have new toys, you have old toys!'

Why (6yr old): We edited file directly on GitHub website, so laptop was old.

How we fixed: git pull origin main --rebase then push again - like syncing your notebook with friend's notebook.

Problem 3: CreateProcess error=87 - Cannot run program ""

What Jenkins/Git said: Jenkins boss robot said 'I don't know where Git tool is!'

Why (6yr old): Jenkins didn't know where git.exe lives.

How we fixed: Manage Jenkins -> Tools -> Git installations -> Add path C:\Program Files\Git\bin\git.exe - like telling robot where hammer is kept.

Problem 4: Failed to connect to repository

What Jenkins/Git said: Jenkins said 'I can't open GitHub book, it's locked!'

Why (6yr old): We needed a key (GitHub Token).

How we fixed: Generate token on GitHub ghp_xxx and add in Jenkins Credentials -> Username with password -> Username afzal-baba, Password = token. Like giving house key to robot.

Problem 5: 401 Unauthorized - SonarQube

What Jenkins/Git said: Toy checker said 'Who are you? I don't know you!'

Why (6yr old): We used fake token sqp_YOUR_TOKEN or SonarCloud token.

How we fixed: Go to <http://localhost:9000> -> My Account -> Security -> Generate token -> Copy real token and put in Jenkinsfile - like showing real ID card.

Problem 6: sonar.organization error

What Jenkins/Git said: Checker said 'Which company you belong to?'

Why (6yr old): We used SonarCloud token which asks for organization, but we have local SonarQube.

How we fixed: Use ONLY token from localhost:9000, not from sonarcloud.io.

Problem 7: Branch master vs main

What Jenkins/Git said: Jenkins searching in master, but we have main

Why (6yr old): GitHub now uses 'main' not 'master'.

How we fixed: Change Branch Specifier from */master to */main in Jenkins config.

Problem 8: Nested folder project-GBGHMVNNEXSONJEN inside itself

What Jenkins/Git said: We accidentally put box inside same box!

Why (6yr old): We cloned project inside itself.

How we fixed: rm -rf nested folder and git rm --cached.

4. Step-by-Step - What We Did From Zero

Step 1: Made folder E:\project-GBGHMVNNEXSONJEN - like making empty factory.

Step 2: Created pom.xml - wrote recipe with Nexus address <http://localhost:8081/repository/maven-snapshots>.

Step 3: Created App.java - simple Hello World program.

Step 4: Created Jenkinsfile - boss robot's 3-step list: Build, Deploy to Nexus, SonarQube check.

Step 5: Setup Maven settings.xml in C:\Users\AFZALBABA\.m2\settings.xml with Nexus username admin and password.

Step 6: Ran mvn clean package - built jar file target/project-GBGHMVNNEXSONJEN-1.0-SNAPSHOT.jar - like first toy made!

Step 7: Ran mvn deploy - put toy on Nexus shelf.

Step 8: Created GitHub repo github.com/afzal-baba/project-GBGHMVNNEXSONJEN and pushed code.

Step 9: Started Jenkins, Nexus, SonarQube services - switched on all machines.

Step 10: Created Jenkins job GBGHMVNNEXSONJEN -> Pipeline script from SCM -> Git URL.

Step 11: Added GitHub token in Jenkins credentials - gave key to boss.

Step 12: Fixed Git tool path in Jenkins -> Told boss where git.exe lives.

Step 13: Fixed Sonar token from localhost:9000 -> Real ID card for checker.

Step 14: Clicked Build Now -> All 3 stages GREEN -> BUILD SUCCESS!

Step 15: Checked Nexus <http://localhost:8081> - jar is there! Checked SonarQube <http://localhost:9000> - report is there!

5. Your Final Jenkinsfile (The Boss List)

```
pipeline { agent any tools { jdk 'jdk21'; maven 'maven3' } stages { stage('Build') { steps { bat 'cd /d E:\project-GBGHMVNNEXSONJEN && mvn clean package -DskipTests' } } stage('Deploy to Nexus') { steps { bat 'cd /d E:\project-GBGHMVNNEXSONJEN && mvn deploy -DskipTests' } } stage('SonarQube Analysis') { steps { bat 'cd /d E:\project-GBGHMVNNEXSONJEN && mvn sonar:sonar -Dsonar.projectKey=project-GBGHMVNNEXSONJEN -Dsonar.host.url=http://localhost:9000 -Dsonar.token=YOUR_REAL_TOKEN' } } }
```

6. Important URLs - Your Factory Addresses

- **GitHub Code:** <https://github.com/afzal-baba/project-GBGHMVNNEXSONJEN>
- **Jenkins Boss:** <http://localhost:8080> -> Job GBGHMVNNEXSONJEN
- **Nexus Shelf:** <http://localhost:8081> -> Browse -> maven-snapshots -> com.gbghmvnnexsonjen
- **SonarQube Checker:** <http://localhost:9000> -> Project project-GBGHMVNNEXSONJEN
- **Local Project:** E:\project-GBGHMVNNEXSONJEN

7. What to Say in Interview (Simple Story)

"I built a full CI/CD pipeline from scratch. I used GitHub for code, Maven to build, Nexus to store artifacts, SonarQube for code quality, and Jenkins to automate everything. I faced issues like Git authentication, Jenkins Git tool path error, Sonar token 401, branch main vs master, and I fixed all by configuring credentials and tools. Final build is GREEN and artifacts are deployed to Nexus and analyzed in SonarQube."

Congratulations! You are now a DevOps Engineer!